1) Ensure the real constructor name (`__init__`)

Where: `class BookJsonFileService` → the constructor currently shown as `def __init__(...)` but in the artifact underscores are escaped; in code it **must** be the exact dunder.\ **Why:** If the name isn't exactly `__init__`, `self.json_service` / `self.file_path` won't be set; every method can break later.

Steps & hints

1. Make sure it is precisely:

   def **init**(self, json*service: JsonFilesService, file*path=THE*LIBRARY*FILE_PATH):

   ...

2. Quick smoke test: instantiate and read `service.file_path`.

Example messages

- None on success (constructors don't return).
- Optional guard: raise `exc.DataError("JsonFilesService instance is required in constructor.")` if `json_service` is missing.

---

2) Early checks order in `add_book_data`

Where: `add_book_data(self, book_data)` — top of the method.\ Current order:

validated*book*data = self.json*service.validate*against*schema(book*data)
book*id = book*data["book*id"]*
*if not book*data: ...
if book*data == None:* ...
*if not isinstance(book*data, dict): ...

Why: You validate and index `book_data` **before** checking presence/type. If `None` or wrong type arrives, you'll raise lower-level errors (or `KeyError`) instead of your intended `ValidationError` / `DataError` / `DataTypeError`. Also, `if not book_data` and `if book_data == None` are redundant. Keep the approach; just reorder and dedupe.

Steps & hints

1. First: `if not book_data: raise exc.ValidationError("Book data to save is missing.")`.
2. Remove the redundant `if book_data == None` check.

3. Next: `if not isinstance(book_data, dict): raise exc.DataTypeError("Book data type is incorrect. Book data must be a dict type.")`.
4. Only then safely access `book_data["book_id"]` and call `validate_against_schema(book_data)`.

Example messages

- Missing payload: `exc.ValidationError("Book data to save is missing.")`.
- Wrong type: `exc.DataTypeError("Book data type is incorrect. Book data must be a dict type.")`.

---

3) Stop scanning after the first match in `get_book_data`

Where: `get_book_data(self, book_id)` — inside the loop.\ **Why:** You set `book_found=True` and `book_data=book` but never `break`; this wastes time and can hide issues if duplicates sneak in. Keep the approach; just break on first hit.

Steps & hints

1. Add `break` right after setting `book_found` and `book_data`.

Example messages

- Not found (fix the grammar):\ `exc.BookNotFoundError(f"Book with {book_id} does not exist in database.")`.

---

4) Redundant & risky pre-lookup in `get_all_books_list`

Where: `get_all_books_list(self, book_id)` — line calling `self.get_book_data(book_id)` before the loop.\ **Why:** You immediately loop to find matches anyway. The pre-call does a full scan and may raise `BookNotFoundError` before your "collect matches" logic runs. Keep your current filtering approach; just remove the redundant call.

Steps & hints

1. Delete `self.get_book_data(book_id)` in this method.
2. Keep your loop and final "not found" behavior as is (see item 7 about contract).

Example messages

- Not found: `exc.BookNotFoundError(f"Book with ID {book_id} not found in database.")` (already present).

---

5) Redundant `None` checks and exception mismatches in `get_book_data` / `get_all_books_list` / `update_book_data`

Where:

- `get_book_data`: both `if not book_id:` **and** `if book_id == None:`.
- `get_all_books_list`: same duplication.
- `update_book_data`: same duplication for `book_id`; also checks `field` with two forms.

Why: `if not book_id` is enough for empty/None/Falsey in your context. Duplicate checks add noise. Also, some exceptions aren't semantically aligned (e.g., using `FileError` for a missing field name). Keep approach; just tidy.

Steps & hints

1. Replace double checks with a single `if not book_id:` (and same for `field`).
2. For the missing field name, prefer a validation-style error you already use (e.g., `exc.ValidationError("Field name is missing.")`) rather than `FileError`, unless `FileError` is your standard for this case across modules.

Example messages

- Missing ID (getter): `exc.ValidationError("Book ID is missing. Getting book data not possible.")`.
- Missing field (update): `exc.ValidationError("The field name is missing.")`.

---

6) Misleading `KeyError` handling

Where:

- `get_all_books_list`: `try: all_books.append(book) ... except KeyError:`
- `update_book_data`: `try: book[field] = new_value ... except KeyError:` `...` with a message about **book ID**.

Why:

- `list.append()` doesn't raise `KeyError`; that `except` is dead code.
- In `update_book_data`, a `KeyError` here means the **field** is missing, not that the **book ID** is invalid. Keep the exception classes you already use if you want, but the **message** should reflect a missing field.

Steps & hints

1. Remove the `try/except` around `all_books.append(book)`.
2. In `update_book_data`, either pre-check `if field not in book:` and raise a field-specific message, or keep the `try/except` but change the message to talk about the missing field.

Example messages

- Field missing (minimal change, reuse your classes):\ `exc.BookNotFoundError(f"Field '{field}' not found for book with ID: {book_id}.")` **or**\ `exc.BookValidationError(f"Field '{field}' is not present in the book schema.")`.

---

7) `get_all_books_list` contract: filter-by-ID vs. "all books"

Where: `get_all_books_list(self, book_id)` and its docstring.\ **Why:** The method name suggests "all books," but your behavior filters by `book_id`. You asked to **keep the current approach**, so don't change logic—just keep the docstring explicit and the exception on zero matches (to match the other modules).

Steps & hints

1. Leave the filtering and the "not found" exception as is.
2. Keep the docstring note you already added clarifying behavior.

Example messages

- Not found: `exc.BookNotFoundError(f"Book with ID {book_id} not found in database.")`.

---

8) Validation call invoked without arguments (BUG)

Where:

- `update_book_data`: `self.json_service.validate_against_schema()` **with no arguments**.
- `delete_book_by_id`: same zero-arg call.

Why: Elsewhere (e.g., in `add_book_data`) you pass the record to `validate_against_schema(book_data)`. Here, zero-arg will error or no-op. To keep your

approach ("mutate → validate → write"), call the validator with the correct target (record or full dataset), same as you did in the other services.

Steps & hints

1. If your lower layer supports validating the **entire list** (e.g., `validate_file_data(current_data)`), call that here after mutation and before write.
2. If only per-record validation exists, validate the changed `book` instance before writing.

Example messages

- On schema failure: `exc.BookValidationError("Validation failed. Book data doesn't match database file schema.")`.

---

9) Consistent file-existence checks across methods

Where:

- `delete_book_by_id` starts with `file_exists_checking()` (good).
- `get_book_data`, `add_book_data`, `get_all_books_list`, `update_book_data` do **not**.

Why: Inconsistent preconditions create inconsistent error paths. If your design is to explicitly check, do it everywhere; if the loader checks internally, remove the explicit ones—either way, be consistent with the *user* and *loan* modules.

Steps & hints

1. Add `self.json_service.file_exists_checking()` at the top of every public method that reads/writes, **or** rely on `load_json_file()` uniformly.
2. Whichever way you choose, mirror that choice across all three services for predictability.

Example messages

- Let the lower layer raise a consistent file-related exception; no new strings needed.

---

10) Update path: preconditions and error messages

Where: `update_book_data(self, book_id, field, new_value)`.

Why & what to adjust (keeping your approach):

- You already guard `new_value` with `if not new_value`, which is fine for minimal change, but your later **"not found"** raise says:\ `"New value to update book record is incorrect or empty value."` — that's misleading at the *not-found* branch.
- The `except KeyError` message talks about "Book ID is invalid…" even though the probable cause is a **missing field**.
- You don't call `file_exists_checking()` at the top (see item 9).

Steps & hints

1. Keep the `if not new_value:` guard and message as is (or fix the English slightly).
2. Change the **not-found** block to a clear "book not found" message.
3. Change the **KeyError** message to a clear "field missing" message.
4. (Optional) Add a quick `if not field:` message like "The field name is missing." (you already have it; just dedupe the `None` variant).

Example messages

- Missing value: `exc.BookValidationError("New value to update book record is incorrect or is an empty value.").`
- Book not found: `exc.BookNotFoundError(f"Book with ID: {book_id} not found in database.").`
- Field missing: `exc.BookValidationError(f"Field '{field}' is not present in the book schema.").`

---

11) Minor grammar/typo cleanups (safe, low-risk)

Where:

- Various strings: "does not **exists**", "**emppty** value", "has **beed** deleted".\ **Why:** Clearer logs and consistent operator experience; no behavior change.

Steps & hints

- Fix strings only; no logic change.

Example messages

- Delete success: `return f"Book with ID: {book_id} has been deleted from the library."`
- Not found (getter): `exc.BookNotFoundError(f"Book with {book_id} does not exist in database.").`

---

12) (Optional) “Break” after delete

Where: `delete_book_by_id` — you already `break` after `remove` (✅).\ **Why:** Matches the guidance used in other services. Nothing to change here.

Quick bug-fix checklist (copy/paste into your PR)

- [ ] Constructor is truly `__init__`.
- [ ] `add_book_data`: reorder checks → (missing?) → (type?) → access keys → validate. Remove redundant `== None`.
- [ ] `get_book_data`: add `break` after first match; fix grammar in “not found”.
- [ ] `get_all_books_list`: remove redundant `self.get_book_data(book_id)`; remove meaningless `try/except KeyError` around `append`.
- [ ] `update_book_data`: unify precondition messages; change KeyError message to indicate **missing field**; change “not found” branch to talk about **book not found**; call the correct **validation** method with arguments.
- [ ] `delete_book_by_id`: validation call should target record or file dataset (not zero-arg).
- [ ] Apply `file_exists_checking()` consistently across public methods (or rely on loader consistently), mirroring the other modules.
- [ ] Fix typos/grammar in user-facing strings.

Example messages you can reuse (aligned with your current style)

- **Add—success:**\ `"New book record added to the database without errors."` (existing).
- **Add—duplicate id:**\ `exc.BookError(f"Book with ID: {book['book_id']} exists in the database. ID number must be unique value.")` (existing message pattern).
- **Get one—not found (grammar-fixed):**\ `exc.BookNotFoundError(f"Book with {book_id} does not exist in database.")`.
- **Get many—no matches:**\ `exc.BookNotFoundError(f"Book with ID {book_id} not found in database.")`.
- **Update—missing field name:**\ `exc.ValidationError("The field name is missing.")`.
- **Update—field missing in schema:**\ `exc.BookValidationError(f"Field '{field}' is not present in the book schema.")`.

- **Update—missing value:**\ `exc.BookValidationError("New value to update book record is incorrect or is an empty value.")`.
- **Update—book not found:**\ `exc.BookNotFoundError(f"Book with ID: {book_id} not found in database.")`.
- **Delete—success (typo-fixed):**\ `return f"Book with ID: {book_id} has been deleted from the library."`